Projects

5th Semester Engineering — 2-Phase Progressive Build (Vanilla JS → Full MERN)

Category A: EdTech & Campus Utilities

1. Dynamic Exam Seating & Conflict Resolver

- **Core Concept**: Generates exam hall seating charts that keep students from the same subject/branch apart to prevent cheating, across multiple rooms with varying capacities.
- **Anti-Cheat Algorithmic Challenge**: A constraint-satisfaction seating algorithm — no two adjacent seats (front/back/left/right) can hold students from the same course code; must gracefully overflow to next room when constraints can't be met in current one.
- **Phase 1 Scope (Vanilla JS + LocalStorage / Public API)**:
  - Admin form to input room grid dimensions, student list (roll no, subject) via JSON upload.
  - DOM-rendered seating grid with color-coding per subject.
  - LocalStorage persistence of last generated layout; regenerate/export as printable table.
- **Phase 2 Scope (Full MERN Upgrade)**:
  - React: drag-to-adjust seat overrides, live conflict highlighting via Context.
  - Express endpoints: `POST /api/exams/:id/generate-seating`, `PUT /api/seating/:id/override`.
  - MongoDB: `Room`, `Student`, `SeatingPlan` schemas with populate linking students to their assigned seat and exam session; JWT-protected admin routes.

2. Adaptive Quiz Engine with Spaced Repetition

- **Core Concept**: A quiz app that adjusts question difficulty and resurfaces previously-missed questions using a spaced-repetition interval algorithm (like Anki).
- **Anti-Cheat Algorithmic Challenge**: Implement a simplified SM-2 spaced-repetition scheduling algorithm — each question gets an "ease factor" and "next review date" recalculated based on correctness and response time.
- **Phase 1 Scope**:
  - Vanilla JS quiz runner pulling questions from a local JSON bank; timer-based scoring.
  - LocalStorage stores per-question ease factor and next-due timestamp; app filters "due today" questions on load.
- **Phase 2 Scope**:
  - React: dashboard visualizing mastery curve per topic (via chart library), controlled quiz-taking flow with useEffect timers.
  - Express: `GET /api/questions/due`, `POST /api/attempts` triggers server-side SM-2 recalculation.
  - MongoDB: `Question`, `User`, `AttemptLog` (populate to compute per-user ease factors); JWT auth per student account.

3. Campus Lost & Found Matching System

- **Core Concept**: Students post "lost" and "found" item reports; system auto-suggests probable matches instead of a plain listing feed.
- **Anti-Cheat Algorithmic Challenge**: A text/attribute similarity scorer — weighted matching on category, color, location-proximity (campus zone graph), and date-window overlap to rank candidate matches, not just keyword search.
- **Phase 1 Scope**:
  - Forms with validated dropdowns (category, color, zone) and free-text description.

    - Vanilla JS computes a similarity score client-side between new post and existing LocalStorage entries; renders ranked suggestions.
- **Phase 2 Scope**:
    - React controlled multi-field form, Context to hold live suggestion state as user types.
    - Express: `POST /api/items`, `GET /api/items/:id/matches` running the scoring algorithm server-side.
    - MongoDB: `Item` schema with geo-zone enum, Multer for photo upload, populate `reportedBy` → `User`; JWT-gated "claim item" workflow with a status state machine (`open → claimed → verified → closed`).

4. Peer-to-Peer Tutoring Slot Negotiator

- **Core Concept**: Students offering tutoring list available time slots; students seeking help request slots — system auto-resolves overlapping requests via a priority/fairness algorithm.
- **Anti-Cheat Algorithmic Challenge**: Interval-overlap detection across tutor availability + a fairness queue (first-come but capped requests per week per student) to auto-approve or auto-reject bookings without double-booking.
- **Phase 1 Scope**:
    - Calendar-like DOM grid built from scratch (no library) showing time slots; form validation prevents selecting already-booked slots.
    - LocalStorage models tutor availability and booking requests.
- **Phase 2 Scope**:
    - React calendar component with controlled state, useEffect polling for slot updates.
    - Express: `POST /api/slots/:id/book` with server-side overlap validation and fairness-cap logic; middleware to reject over-quota users.
    - MongoDB: `TutorProfile`, `Slot`, `Booking` with populate chains; JWT roles (`tutor`/`student`).

5. Multi-Section Timetable Clash Detector

- **Core Concept**: Students build a personal timetable by picking course sections; app detects and blocks time-overlap clashes across chosen sections.
- **Anti-Cheat Algorithmic Challenge**: Interval-tree/overlap-detection logic across a variable number of weekly recurring time blocks, plus a "suggest alternate section" resolver when a clash is found.
- **Phase 1 Scope**:
  - Vanilla JS renders a weekly grid; selecting a section runs an overlap check against already-picked sections using array iteration, not brute string compare.
  - Validation blocks add-to-timetable with a clear conflict message; LocalStorage saves the finalized timetable.
- **Phase 2 Scope**:
  - React: drag list of available sections, live-updating grid via state; Context holds selected course list.
  - Express: `GET /api/courses/:id/sections`, `POST /api/timetable/validate` doing server-side clash detection (client can't be trusted).
  - MongoDB: `Course` → `Section` (populate), `Timetable` per user; JWT auth.

6. Skill-Based Team Formation Tool for Hackathons

- **Core Concept**: Students list their skills; organizer auto-generates balanced teams so each team has a spread of skill categories (frontend/backend/design/etc.).
- **Anti-Cheat Algorithmic Challenge**: A greedy bin-packing/balancing algorithm that distributes students across N teams to minimize skill-category imbalance — students must design and justify the balancing heuristic.
- **Phase 1 Scope**:
  - Skill-tag multi-select form with validation (min/max tags).
  - Vanilla JS balancing algorithm run on a LocalStorage array of participants; renders resulting team cards.
- **Phase 2 Scope**:

- React: organizer dashboard with re-shuffle button (re-runs algorithm), manual drag-override between teams.
- Express: `POST /api/hackathons/:id/generate-teams` server-side balancing.
- MongoDB: `Participant`, `Team`, `Hackathon` schemas with populate; JWT for organizer-only regeneration rights.

7. Library Seat & Resource Reservation with Priority Queue

- **Core Concept**: Reserve library seats or shared resources (projectors, discussion rooms) where final-year/thesis students get priority during exam weeks.
- **Anti-Cheat Algorithmic Challenge**: A weighted priority-queue booking system — priority score computed from user role, booking history (no-show penalty), and time-to-exam proximity, not simple FCFS.
- **Phase 1 Scope**:
  - Seat-map grid built with DOM manipulation; booking form with time-slot validation against LocalStorage bookings.
  - Simple penalty counter stored in LocalStorage for no-shows.
- **Phase 2 Scope**:
  - React seat map with live state, Context for current user's priority score.
  - Express: `POST /api/reservations` computing priority server-side and auto-bumping lower-priority pending requests.
  - MongoDB: `User` (with `noShowCount`), `Reservation`, `Resource`; JWT auth, cron-like cleanup of expired holds.

8. Assignment Plagiarism Similarity Scorer

- **Core Concept**: Students submit text assignments; tool flags pairs of submissions with high textual similarity for instructor review.
- **Anti-Cheat Algorithmic Challenge**: Implement a hand-coded n-gram/shingling + Jaccard similarity algorithm (no external plagiarism API) to score every submission pair and rank suspicious ones.

- **Phase 1 Scope**:
  - Textarea submission form with word-count validation.
  - Vanilla JS tokenizes text into n-grams and computes pairwise Jaccard similarity across LocalStorage-stored submissions; renders a similarity matrix/heatmap via DOM.
- **Phase 2 Scope**:
  - React: instructor dashboard listing flagged pairs with a similarity threshold slider (state-driven re-filtering).
  - Express: `POST /api/submissions`, `GET /api/assignments/:id/similarity-report` running the n-gram algorithm server-side on submission text.
  - MongoDB: `Assignment`, `Submission`, `SimilarityReport`; Multer for optional file upload variant; JWT roles (`student` / `instructor`).

9. Course Prerequisite Dependency Tracker

- **Core Concept**: Visualizes and validates a student's course registration against a prerequisite dependency graph, blocking illegal registrations.
- **Anti-Cheat Algorithmic Challenge**: Build a directed-graph traversal (DFS/topological check) to validate prerequisite chains and detect circular or missing dependencies before allowing registration.
- **Phase 1 Scope**:
  - Vanilla JS renders course tree from a JSON dependency file; registration form validates chosen course against completed-courses list in LocalStorage using graph traversal.
  - Clear inline error messaging on validation forms.
- **Phase 2 Scope**:
  - React: interactive dependency tree/graph visualization (state-driven expand/collapse).
  - Express: `POST /api/registrations/validate` server-side graph check.
  - MongoDB: `Course` with `prerequisites: [ObjectId]` self-referencing populate, `StudentRecord`; JWT auth.

10. Campus Event Budget & Sponsorship Tracker

- **Core Concept**: Event committees track multi-category budgets (venue, food, marketing) against sponsorship inflows, with automatic overspend alerts and category reallocation logic.
- **Anti-Cheat Algorithmic Challenge**: A running-balance reallocation engine — when one category overspends, system computes proportional deduction suggestions from underspent categories to auto-balance the budget.
- **Phase 1 Scope**:
  - Category-based expense/income entry form with numeric validation.
  - Vanilla JS reduce()-based running totals per category; LocalStorage persistence; renders a live budget bar chart drawn on `<canvas>`.
- **Phase 2 Scope**:
  - React: controlled multi-category budget form, Context for global budget state, canvas or SVG chart component.
  - Express: CRUD for `Transaction`, `POST /api/budgets/:id/rebalance` implementing the reallocation algorithm.
  - MongoDB: `Event`, `Budget`, `Transaction` (populate); JWT roles (`treasurer`/`member`).

Category B: Healthcare, Fitness & Personal Management

11. Medication Dosage & Interaction Scheduler

- **Core Concept**: Tracks multiple medications per user and builds a daily dosage schedule while flagging known drug-interaction conflicts and dosage-spacing violations.
- **Anti-Cheat Algorithmic Challenge**: A time-window conflict resolver — checks minimum spacing rules between specific drug pairs (from a rules table) and auto-reschedules conflicting doses across the day.
- **Phase 1 Scope**:

    - Medication entry form (name, dose, frequency) with validation; static JSON interaction rules file.
    - Vanilla JS builds a 24-hour schedule array, checks pairwise spacing against rules, flags conflicts in the DOM; LocalStorage persistence.
- **Phase 2 Scope**:
    - React: daily timeline component (state-driven), caregiver view via Context.
    - Express: `POST /api/schedule/generate` running conflict-resolution server-side against an `InteractionRule` collection.
    - MongoDB: `Medication`, `InteractionRule`, `ScheduleEntry` (populate); JWT auth, Multer for prescription image upload.

12. Macro-Nutrient Reverse Calculator (Meal Planner)

- **Core Concept**: User sets a target macro split (protein/carb/fat) and calorie goal; app reverse-engineers meal combinations from a food database that hit the target within a tolerance band.
- **Anti-Cheat Algorithmic Challenge**: A constraint-solving combination search (subset-sum-like) across food items to find combinations that fit calorie + macro tolerance — not a simple sum/display.
- **Phase 1 Scope**:
    - Form to set daily macro targets; food-search UI against a local JSON food database.
    - Vanilla JS combination-search algorithm suggests meal sets; LocalStorage saves favorite combos and daily logs.
- **Phase 2 Scope**:
    - React: multi-step controlled form for goal setup, live macro-remaining ring/bar via state.
    - Express: `POST /api/mealplan/suggest` running the combination algorithm against MongoDB food data.
    - MongoDB: `FoodItem`, `MealLog`, `UserGoal`; JWT auth.

13. Symptom-Based Triage Priority Queue (Non-Diagnostic)

- **Core Concept**: A campus/clinic front-desk tool where patients self-report symptoms and severity; system computes a triage priority score to order the waiting queue (explicitly non-diagnostic, educational).
- **Anti-Cheat Algorithmic Challenge**: A weighted scoring algorithm combining symptom severity, duration, and vital-sign flags into a triage score, plus a dynamic priority-queue re-ordering as new patients arrive.
- **Phase 1 Scope**:
  - Symptom checklist form with severity sliders; vanilla JS computes triage score and inserts patient into a sorted LocalStorage queue (custom insertion-sort logic, not `Array.sort` alone — must justify tie-breaking).
  - DOM renders live-updating queue.
- **Phase 2 Scope**:
  - React: receptionist dashboard with real-time queue (useEffect polling), patient controlled form.
  - Express: `POST /api/queue/add` recalculating and re-sorting server-side; `PUT /api/queue/:id/serve`.
  - MongoDB: `Patient`, `QueueEntry` with `triageScore`; JWT roles (`staff`/`patient`).

14. Progressive Overload Workout Planner

- **Core Concept**: Generates a multi-week strength-training plan that auto-increments weights/reps based on the user's logged performance (progressive overload logic).
- **Anti-Cheat Algorithmic Challenge**: A rule-based progression engine — if last session's reps/weight met threshold, auto-increase next session's target by a computed percentage; if failed twice, auto-deload. This is a state machine, not static data.
- **Phase 1 Scope**:
  - Workout log form (exercise, sets, reps, weight) with validation.

    - Vanilla JS progression algorithm computes next session's targets from LocalStorage history; renders a progress chart on canvas.
- **Phase 2 Scope**:
    - React: workout logging UI with controlled inputs, progression state shown via Context.
    - Express: `POST /api/workouts/log` triggering server-side progression recalculation, `GET /api/plan/next`.
    - MongoDB: `Exercise`, `WorkoutLog`, `ProgressionState`; JWT auth.

15. Blood Donation Eligibility & Matching System

- **Core Concept**: Tracks donor eligibility windows (based on last donation date, blood type compatibility) and matches donors to urgent requests.
- **Anti-Cheat Algorithmic Challenge**: A blood-type compatibility matrix lookup combined with date-based eligibility calculation (donation cooldown periods differ by donation type) and proximity-based ranking of matched donors.
- **Phase 1 Scope**:
    - Donor registration form with blood-type dropdown and last-donation date validation.
    - Vanilla JS computes eligibility countdown and compatibility matches against a static compatibility matrix; LocalStorage stores donor list.
- **Phase 2 Scope**:
    - React: request broadcast UI, donor-match list with live eligibility state.
    - Express: `POST /api/requests`, `GET /api/requests/:id/matches` running compatibility + eligibility logic server-side.
    - MongoDB: `Donor`, `DonationRecord`, `Request` (populate); JWT auth, geo-zone field for proximity ranking.

16. Sleep Cycle & Circadian Rhythm Optimizer

- **Core Concept**: Given desired wake time and sleep-cycle length (90 min), suggests optimal bedtimes; also analyzes logged sleep data for consistency

scoring.

- **Anti-Cheat Algorithmic Challenge**: Backward time-arithmetic across 90-minute sleep cycles factoring in fall-asleep latency, plus a consistency/variance score computed from logged sleep history (standard deviation calc, hand-coded).
- **Phase 1 Scope**:
  - Form for wake-time + latency input; vanilla JS computes multiple candidate bedtimes.
  - Sleep log entries stored in LocalStorage; variance/consistency score computed and displayed.
- **Phase 2 Scope**:
  - React: controlled time-picker form, consistency trend chart via state.
  - Express: `POST /api/sleep/log`, `GET /api/sleep/analytics` computing variance server-side.
  - MongoDB: `SleepLog`, `UserPreference`; JWT auth.

17. Chronic Condition Habit Streak Tracker with Relapse Logic

- **Core Concept**: Tracks daily adherence to health habits (medication, exercise, diet) for chronic-condition management, with nuanced streak logic that accounts for “grace days” and relapse patterns.
- **Anti-Cheat Algorithmic Challenge**: A streak-calculation state machine that isn’t a naive counter — must handle grace-day allowances, partial-completion weighting, and relapse-triggered streak resets with a recovery bonus mechanic.
- **Phase 1 Scope**:
  - Daily check-in form with partial-completion options; vanilla JS streak engine processes LocalStorage history array to compute current streak, best streak, grace days used.
  - Calendar heatmap rendered via DOM (GitHub-style).
- **Phase 2 Scope**:

- React: calendar heatmap component (state-driven), controlled check-in form.
- Express: `POST /api/habits/:id/checkin` recalculating streak server-side; `GET /api/habits/:id/stats`.
- MongoDB: `Habit`, `CheckIn` array (or subdocs), `StreakState`; JWT auth.

18. Multi-Doctor Appointment Conflict Scheduler

- **Core Concept**: A clinic scheduler across multiple doctors/rooms where patients book appointments; system prevents double-booking and suggests next available slot per doctor specialty.
- **Anti-Cheat Algorithmic Challenge**: Multi-resource interval-overlap checking (doctor AND room availability simultaneously) with a "next available slot" search algorithm across a rolling week.
- **Phase 1 Scope**:
  - Booking form with doctor/specialty dropdown; vanilla JS overlap-check against LocalStorage-stored appointments for both doctor and room resources.
  - "Suggest next slot" button running a linear search over open windows.
- **Phase 2 Scope**:
  - React: weekly calendar grid with controlled booking flow.
  - Express: `POST /api/appointments` with dual-resource conflict validation; `GET /api/doctors/:id/next-available`.
  - MongoDB: `Doctor`, `Room`, `Appointment` (populate both); JWT roles (`patient`/`doctor`/`admin`).

19. Caregiver Task Rotation Scheduler

- **Core Concept**: Families/caregivers of an elderly or ill relative rotate daily care tasks (meds, meals, checkups); system fairly auto-assigns tasks avoiding overload of any one caregiver.
- **Anti-Cheat Algorithmic Challenge**: A fair-rotation load-balancing algorithm tracking cumulative task-hours per caregiver and assigning new tasks to

whoever has the lowest running load (weighted round-robin, not plain round-robin).

- **Phase 1 Scope**:
  - Task-definition form (type, estimated duration, recurrence); vanilla JS load-balancer assigns tasks across a LocalStorage caregiver list and renders a weekly roster.
- **Phase 2 Scope**:
  - React: roster board with drag-to-swap (still validated against load-balance rules), Context for caregiver load state.
  - Express: `POST /api/tasks/assign` running the load-balancing algorithm server-side.
  - MongoDB: `Caregiver`, `Task`, `Assignment` (populate); JWT auth per family group.

20. Mental Wellness Mood-Pattern Correlator

- **Core Concept**: Users log daily mood + lifestyle factors (sleep, exercise, screen time); app surfaces correlations between factors and mood trends over time.
- **Anti-Cheat Algorithmic Challenge**: Hand-coded correlation coefficient calculation (Pearson-style) between logged factors and mood score across the stored history — real statistics, not just a line chart.
- **Phase 1 Scope**:
  - Daily mood + factor logging form with validated numeric ranges.
  - Vanilla JS computes correlation coefficients across LocalStorage history; renders results with basic canvas scatter plot.
- **Phase 2 Scope**:
  - React: logging form with sliders (controlled), insights dashboard via state/Context.
  - Express: `GET /api/moods/correlations` computing coefficients server-side across a user's full history.

- MongoDB: `MoodLog`; JWT auth; optional aggregation pipeline for correlation prep.

Category C: FinTech, Smart Logistics & Utility Tools

21. Dynamic Split-Bill Settlement Optimizer

- **Core Concept**: Group expense tracker (like a trip fund) that computes the *minimum number of transactions* needed to settle all debts among members, not just raw balances.
- **Anti-Cheat Algorithmic Challenge**: A debt-simplification/graph-minimization algorithm (greedy max-creditor/max-debtor matching) to reduce a tangled web of who-owes-whom into the fewest possible payments.
- **Phase 1 Scope**:
  - Expense entry form (payer, amount, split-among selection) with validation.
  - Vanilla JS computes net balances then runs the debt-simplification algorithm; LocalStorage stores group + expenses; renders settlement plan.
- **Phase 2 Scope**:
  - React: group expense UI with controlled split-selection, settlement-plan view via Context.
  - Express: `POST /api/groups/:id/settle` running simplification algorithm server-side.
  - MongoDB: `Group`, `Expense`, `Settlement` (populate members); JWT auth.

22. Subscription Overlap & Cost Leak Detector

- **Core Concept**: Users log recurring subscriptions; app detects redundant/overlapping services (e.g., two music apps) and projects annualized “cost leaks.”
- **Anti-Cheat Algorithmic Challenge**: A category-overlap detection algorithm plus billing-cycle normalization (converting weekly/monthly/yearly costs to a common unit) to compute true annual spend and flag redundancy.

- **Phase 1 Scope**:
  - Subscription entry form (name, category, cost, billing cycle) with validation.
  - Vanilla JS normalizes all costs to monthly-equivalent, groups by category to flag overlaps; LocalStorage persistence; renders a spend-breakdown chart.
- **Phase 2 Scope**:
  - React: subscription manager with controlled forms, insights panel via state.
  - Express: `GET /api/subscriptions/analysis` running normalization + overlap detection server-side.
  - MongoDB: `Subscription`; JWT auth.

23. Multi-Tier Dynamic Pricing Engine for Rentals

- **Core Concept**: A rental-listing platform (equipment, rooms, vehicles) where price per day dynamically changes based on demand tier, booking lead-time, and duration discounts.
- **Anti-Cheat Algorithmic Challenge**: A multi-factor dynamic-pricing formula combining base price, demand multiplier (based on current booking density), early-bird/last-minute adjustments, and tiered duration discounts — computed live, not hardcoded.
- **Phase 1 Scope**:
  - Listing + booking form with date-range validation.
  - Vanilla JS pricing engine computes final price from multiple factors stored in LocalStorage; live price preview updates as dates change.
- **Phase 2 Scope**:
  - React: controlled date-range picker with live price computation via useEffect.
  - Express: `POST /api/listings/:id/quote` running the full pricing engine server-side (never trust client price).

- MongoDB: `Listing`, `Booking` with computed `finalPrice` field, demand tracking; JWT auth.

24. Vehicle Route & Fuel Cost Optimizer

- **Core Concept**: Given multiple delivery/pickup stops, computes an optimized visiting order to minimize total distance/fuel cost (simplified Traveling Salesman heuristic).
- **Anti-Cheat Algorithmic Challenge**: Implement a nearest-neighbor or 2-opt heuristic for route optimization across student-entered stop coordinates — genuine algorithmic work, not a map API call.
- **Phase 1 Scope**:
  - Form to add stops (name + lat/lng or address); vanilla JS computes distance matrix (Haversine formula) and runs nearest-neighbor heuristic; renders ordered route list.
  - LocalStorage saves stop sets.
- **Phase 2 Scope**:
  - React: stop-list builder with controlled inputs, route visualization (simple SVG map or ordered list with distances).
  - Express: `POST /api/routes/optimize` running the heuristic server-side for larger stop sets.
  - MongoDB: `RoutePlan`, `Stop`; JWT auth.

25. Inventory Reorder Point & Demand Forecasting Tool

- **Core Concept**: Small-business inventory tool that predicts when to reorder stock based on historical sales velocity, not just a static low-stock alert.
- **Anti-Cheat Algorithmic Challenge**: A moving-average demand forecast calculation combined with a reorder-point formula (lead time × average demand + safety stock) computed from logged sales data.
- **Phase 1 Scope**:
  - Sales-entry form; vanilla JS computes moving average and reorder point from LocalStorage sales history; flags items needing reorder in the DOM.

- **Phase 2 Scope**:
  - React: inventory dashboard with controlled sales-entry form, reorder alerts via Context.
  - Express: `GET /api/inventory/forecast` computing moving averages + reorder points server-side.
  - MongoDB: `Product`, `SaleRecord`; JWT auth, Multer for product image upload.

26. Peer Lending Circle (Chit Fund) Simulator

- **Core Concept**: Simulates a rotating savings/chit-fund group where members contribute monthly and one member "wins" the pot each cycle via bidding — app manages the full cycle logic.
- **Anti-Cheat Algorithmic Challenge**: A cycle-based state machine tracking contributions, bid amounts, dividend distribution to non-winners, and eligibility rules (a member can't win twice) across N cycles.
- **Phase 1 Scope**:
  - Group setup + monthly contribution/bid entry forms with validation.
  - Vanilla JS cycle engine processes bids, computes dividends, updates LocalStorage state machine; renders cycle history table.
- **Phase 2 Scope**:
  - React: cycle dashboard with controlled bid-entry, member-eligibility state via Context.
  - Express: `POST /api/circles/:id/bid`, `POST /api/circles/:id/close-cycle` running the full cycle logic server-side.
  - MongoDB: `Circle`, `Member`, `Cycle`, `Bid` (populate chains); JWT auth.

27. Currency Arbitrage & Conversion Fee Calculator

- **Core Concept**: Compares multi-hop currency conversion paths (e.g., INR→USD→EUR vs INR→EUR direct) factoring in each service's fee structure to find the cheapest real path.

- **Anti-Cheat Algorithmic Challenge**: A graph-based shortest-path (Dijkstra-style, minimizing cost not distance) across currency-pair exchange rates and fee nodes, using a public exchange-rate API for live rates.
- **Phase 1 Scope**:
  - Form for source/target currency + amount; vanilla JS fetches live rates via Fetch API, builds a small currency graph, and runs shortest-cost-path logic across 3–4 hop options.
  - LocalStorage caches recent rate lookups.
- **Phase 2 Scope**:
  - React: controlled conversion form with path visualization.
  - Express: `GET /api/convert/best-path` running the graph algorithm server-side, caching external API responses.
  - MongoDB: `RateCache`, `ConversionHistory`; JWT auth (optional, for saved history).

28. Warehouse Slot Allocation & Packing Optimizer

- **Core Concept**: Given incoming items with dimensions and a warehouse's shelf/slot capacity, assigns optimal storage slots to minimize wasted space.
- **Anti-Cheat Algorithmic Challenge**: A bin-packing heuristic (first-fit-decreasing) matching item volume/dimensions to available slot capacities, with re-allocation logic when items are removed.
- **Phase 1 Scope**:
  - Item entry form (dimensions/volume) and slot-definition setup; vanilla JS bin-packing algorithm assigns items to slots from LocalStorage warehouse map; renders a visual slot-occupancy grid.
- **Phase 2 Scope**:
  - React: warehouse grid visualization with controlled item-intake form.
  - Express: `POST /api/warehouse/allocate` running the packing algorithm server-side.
  - MongoDB: `Slot`, `Item`, `Allocation`; JWT auth.

29. Recurring Expense Anomaly Detector

- **Core Concept**: Analyzes a user's transaction history to detect anomalous spending (unusual amount or frequency spikes) compared to their own historical baseline.
- **Anti-Cheat Algorithmic Challenge**: A statistical anomaly-detection routine — compute mean/standard deviation per spending category and flag transactions beyond a z-score threshold, hand-coded (no ML library).
- **Phase 1 Scope**:
  - Transaction entry/import form (CSV or manual); vanilla JS computes per-category baseline stats from LocalStorage history and flags outliers in the DOM.
- **Phase 2 Scope**:
  - React: transaction dashboard with controlled filters, anomaly alerts via state.
  - Express: `GET /api/transactions/anomalies` computing z-scores server-side per category per user.
  - MongoDB: `Transaction`, `CategoryBaseline` (recomputed periodically); JWT auth.

30. Freelancer Multi-Client Invoice & Tax Slab Calculator

- **Core Concept**: Freelancers log billable hours/projects across multiple clients; app auto-generates invoices and computes tax liability using tiered/slab-based tax rules.
- **Anti-Cheat Algorithmic Challenge**: A progressive tax-slab calculation engine (different rates apply to different income bands cumulatively) plus multi-currency/multi-client invoice aggregation logic.
- **Phase 1 Scope**:
  - Time/project entry form with validation; vanilla JS computes invoice totals and applies slab-based tax calculation from a configurable rates JSON; LocalStorage stores client + invoice history.

- **Phase 2 Scope**:
  - React: invoice builder with controlled line-items, tax-summary panel via Context.
  - Express: `POST /api/invoices/generate` computing slab tax server-side; PDF-export-ready data structure.
  - MongoDB: `Client`, `Project`, `Invoice`, `TaxSlabConfig`; JWT auth, Multer for logo/attachment upload.

Category D: Creative Media, Collaboration & Niche Dashboards

31. Collaborative Pixel-Art Canvas with Version History

- **Core Concept**: A shared pixel-art grid where multiple users place pixels; app maintains an undo-able version history and shows a replay of how the art was built.
- **Anti-Cheat Algorithmic Challenge**: A custom canvas state-diffing/undo-stack implementation — each pixel change is a delta object; replay reconstructs the grid by replaying deltas in order (not just storing full snapshots).
- **Phase 1 Scope**:
  - `<canvas>` based pixel grid built with vanilla JS event handling (click/drag to paint); LocalStorage stores the delta history array; “replay” button re-renders frame by frame.
- **Phase 2 Scope**:
  - React canvas component with controlled color-palette state, Context for current session’s delta stream.
  - Express + Socket-less polling (or note for future WebSocket upgrade): `POST /api/canvas/:id/pixel`, `GET /api/canvas/:id/history`.
  - MongoDB: `Canvas`, `PixelDelta` (ordered array/collection); JWT auth per contributor.

32. Dynamic Story Branching Engine (Choose-Your-Own-Adventure Builder)

- **Core Concept**: Lets authors build branching interactive stories (node-based) and lets readers play through them; tracks which paths readers take.
- **Anti-Cheat Algorithmic Challenge**: A directed-graph story engine — nodes with multiple choice-edges, validation to prevent orphaned/unreachable nodes, and path-analytics (most/least visited branches) computed via traversal.
- **Phase 1 Scope**:
  - Vanilla JS story-node editor (add node, add choices linking to other nodes) stored as a JSON graph in LocalStorage; "play mode" traverses the graph based on reader clicks.
  - Validation warns about orphaned nodes.
- **Phase 2 Scope**:
  - React: node-graph editor (state-driven), separate reader-mode component using React Router.
  - Express: `POST /api/stories/:id/nodes`, `GET /api/stories/:id/analytics` computing path-traversal stats.
  - MongoDB: `Story`, `Node` (self-referencing choice edges), `ReadSession`; JWT auth for authors.

33. Music Playlist Mood-Transition Sequencer

- **Core Concept**: Auto-orders a playlist so songs flow smoothly by mood/energy/tempo transition instead of random or alphabetical order.
- **Anti-Cheat Algorithmic Challenge**: A sequencing algorithm that treats songs as nodes with tempo/mood-distance "edges," solving a path that minimizes abrupt transitions (nearest-neighbor-style ordering across multiple attributes).
- **Phase 1 Scope**:
  - Song entry form (or pull metadata from a public music API) capturing tempo/mood/energy tags; vanilla JS computes the sequencing order from

LocalStorage playlist data; renders the ordered playlist with transition-smoothness score.

- **Phase 2 Scope**:
  - React: playlist builder with controlled tag input, drag-reorder override, live "smoothness score" via state.
  - Express: `POST /api/playlists/:id/sequence` running the sequencing algorithm server-side.
  - MongoDB: `Song`, `Playlist` (ordered array); JWT auth.

34. Real-Time Kanban with Dependency-Based Task Locking

- **Core Concept**: A Kanban board where tasks can depend on other tasks — a task can't move to "In Progress" until its dependencies are marked "Done."
- **Anti-Cheat Algorithmic Challenge**: A dependency-graph validation engine (cycle detection + lock/unlock state propagation) that re-evaluates all dependent tasks' lock status whenever any task's status changes.
- **Phase 1 Scope**:
  - Vanilla JS drag-and-drop Kanban board (built without a library) with dependency-selection dropdown per task; LocalStorage stores tasks + dependency edges; movement blocked with a clear message if dependencies aren't met.
- **Phase 2 Scope**:
  - React: drag-and-drop board (state-driven columns), Context for dependency-graph state and lock propagation.
  - Express: `PUT /api/tasks/:id/status` server-side validating and cascading lock/unlock to dependents.
  - MongoDB: `Board`, `Task` with `dependsOn: [ObjectId]` (populate); JWT auth per board member.

35. Polling System with Ranked-Choice Vote Tabulation

- **Core Concept**: Instead of simple single-choice polls, supports ranked-choice voting with instant-runoff tabulation to determine a winner.

- **Anti-Cheat Algorithmic Challenge**: A hand-coded instant-runoff voting (IRV) algorithm — eliminate lowest-ranked option each round, redistribute votes to next preference, repeat until majority winner emerges.
- **Phase 1 Scope**:
  - Drag-to-rank voting form (vanilla JS drag-and-drop or up/down buttons); vanilla JS IRV tabulation engine runs on LocalStorage-stored ballots; renders round-by-round elimination results.
- **Phase 2 Scope**:
  - React: controlled ranked-choice ballot component, live results-by-round visualization.
  - Express: `POST /api/polls/:id/vote`, `GET /api/polls/:id/tabulate` running IRV server-side.
  - MongoDB: `Poll`, `Ballot` (ranked array), `TabulationRound` log; JWT auth (one vote per user enforced).

36. Recipe Scaling & Substitution Engine

- **Core Concept**: Scales recipes to any serving size while correctly handling non-linear ingredients (e.g., spices don't scale linearly, baking leaveners need special ratios) and suggests substitutions for missing ingredients.
- **Anti-Cheat Algorithmic Challenge**: A non-linear scaling engine with per-ingredient scaling rules (some linear, some logarithmic/capped) plus a substitution-matching lookup with ratio conversion (e.g., baking soda ↔ baking powder conversion math).
- **Phase 1 Scope**:
  - Recipe entry form with per-ingredient scaling-type tags; vanilla JS scaling engine computes adjusted quantities for a target serving size; LocalStorage saves recipes; substitution suggestions pulled from a static rules JSON.
- **Phase 2 Scope**:
  - React: controlled serving-size slider with live-recalculated ingredient list via useEffect.

- Express: `GET /api/recipes/:id/scale?servings=N` computing server-side, `GET /api/ingredients/:id/substitutes`.
- MongoDB: `Recipe`, `Ingredient` (with `scalingType`), `SubstitutionRule`; JWT auth, Multer for recipe photo.

37. Photo Contest Judging with Weighted Scoring Rounds

- **Core Concept**: Manages a multi-round photo contest where judges score on multiple weighted criteria (creativity, technical, theme-fit), aggregating into a final ranked leaderboard.
- **Anti-Cheat Algorithmic Challenge**: A weighted multi-criteria scoring aggregation engine that normalizes scores across judges (to offset a "harsh" vs "lenient" judge bias) before computing final rankings.
- **Phase 1 Scope**:
  - Entry submission form (with image preview via FileReader) and judge-scoring form with weighted sliders; vanilla JS computes normalized weighted scores from LocalStorage judge entries; renders a live leaderboard.
- **Phase 2 Scope**:
  - React: judge scoring UI (controlled sliders), public leaderboard component with real-time state updates.
  - Express: `POST /api/entries/:id/score`, `GET /api/contests/:id/leaderboard` running normalization + aggregation server-side.
  - MongoDB: `Contest`, `Entry`, `Score` (populate judge + entry); Multer for photo upload; JWT roles (`judge` / `participant`).

38. Podcast Chapter & Timestamp Annotation Tool

- **Core Concept**: Lets creators upload/link an audio file and mark timestamped chapters/annotations; listeners can jump to chapters and see synced notes as audio plays.
- **Anti-Cheat Algorithmic Challenge**: A time-synced annotation engine — binary-search lookup to find the "current active annotation" as

`audio.currentTime` updates on every tick, plus overlap-validation so chapters can't have conflicting time ranges.

- **Phase 1 Scope**:
  - Vanilla JS `<audio>` element with custom controls; annotation form (timestamp + note) with overlap validation; LocalStorage stores chapter list; DOM highlights the active chapter as audio plays (via `timeupdate` event + binary search over sorted chapters).
- **Phase 2 Scope**:
  - React: audio player component with controlled annotation editor, Context tracking active chapter.
  - Express: `POST /api/episodes/:id/chapters` with server-side overlap validation.
  - MongoDB: `Episode`, `Chapter` (sorted by timestamp); Multer for audio file upload; JWT auth for creators.

39. Multiplayer Turn-Based Strategy Game State Manager

- **Core Concept**: A simple turn-based grid strategy game (e.g., territory capture) where each move must be validated against game rules and turn order.
- **Anti-Cheat Algorithmic Challenge**: A full game-state machine — turn-order enforcement, move-legality validation (based on adjacency/resource rules), win-condition checking after every move, all server-authoritative to prevent client-side cheating.
- **Phase 1 Scope**:
  - Vanilla JS grid-based game board with click-to-move; move validation logic (adjacency rules) run entirely client-side against a LocalStorage game-state object; turn-switching logic.
- **Phase 2 Scope**:
  - React: game board component (state-driven re-render each move), Context for current game session.
  - Express: `POST /api/games/:id/move` — **all validation moved server-side** (this is the key MERN-upgrade lesson: never trust client game state); win-

condition check triggers game-close.

- MongoDB: `Game` (board state, turn, players), `MoveLog`; JWT auth, two-player session matching.

40. Freelance Portfolio with Dynamic Case-Study Analytics Dashboard

- **Core Concept**: A portfolio site where each project entry auto-computes and displays engagement analytics (views, time-on-page, referrer sources) and a dynamically generated “skill frequency” chart from all listed projects.
- **Anti-Cheat Algorithmic Challenge**: A tag-frequency/word-frequency aggregation algorithm across all projects (to build the skill chart) plus session-based analytics tracking logic (deduplicating repeat views within a time window) hand-coded without an analytics SDK.
- **Phase 1 Scope**:
  - Vanilla JS portfolio pages with a project-entry admin form; view-tracking via LocalStorage/SessionStorage with time-window deduplication logic; skill-frequency computed via `reduce()` across all projects and rendered as a bar chart on canvas.
- **Phase 2 Scope**:
  - React: public portfolio (React Router pages), admin dashboard with controlled project-entry form, analytics charts via state.
  - Express: `POST /api/analytics/view` with server-side dedup logic (IP/session-hash + time window), `GET /api/analytics/skills-frequency` aggregation endpoint.
  - MongoDB: `Project`, `ViewLog`, aggregation pipeline for skill-frequency; JWT auth for admin-only project management; Multer for project images.
